```
end
```

4.4.2.2 :autosave

If you set the `:autosave` option to `true`, Rails will save any loaded members and destroy members that are marked for destruction whenever you save the parent object.

4.4.2.3 :class_name

If the name of the other model cannot be derived from the association name, you can use the `:class_name` option to supply the model name. For example, if a part has many assemblies, but the actual name of the model containing assemblies is `Gadget`, you'd set things up this way:

```
class Parts < ActiveRecord::Base
  has_and_belongs_to_many :assemblies, class_name: "Gadget"
end
```

4.4.2.4 :foreign_key

By convention, Rails assumes that the column in the join table used to hold the foreign key pointing to this model is the name of this model with the suffix `_id` added. The `:foreign_key` option lets you set the name of the foreign key directly:

```
class User < ActiveRecord::Base
  has_and_belongs_to_many :friends,
      class_name: "User",
      foreign_key: "this_user_id",
      association_foreign_key: "other_user_id"
end
```

4.4.2.5 :join_table

If the default name of the join table, based on lexical ordering, is not what you want, you can use the `:join_table` option to override the default.

4.4.2.6 :validate

If you set the `:validate` option to `false`, then associated objects will not be validated whenever you save this object. By default, this is `true`: associated objects will be validated when this object is saved.

4.4.3 Scopes for has_and_belongs_to_many

There may be times when you wish to customize the query used by `has_and_belongs_to_many`. Such customizations can be achieved via a scope block. For example:

```
class Parts < ActiveRecord::Base
  has_and_belongs_to_many :assemblies, -> { where active: true }
end
```

You can use any of the standard querying methods inside the scope block. The following ones are discussed below:

- `where`
- `extending`
- `group`
- `includes`
- `limit`
- `offset`
- `order`
- `readonly`
- `select`
- `uniq`

4.4.3.1 where

The `where` method lets you specify the conditions that the associated object must meet.

```
class Parts < ActiveRecord::Base
  has_and_belongs_to_many :assemblies,
    -> { where "factory = 'Seattle'" }
end
```

You can also set conditions via a hash: